

Peter Müller

AUTOMATING SEPARATION LOGIC PROOFS WITH VIPER

Modular reasoning about procedures

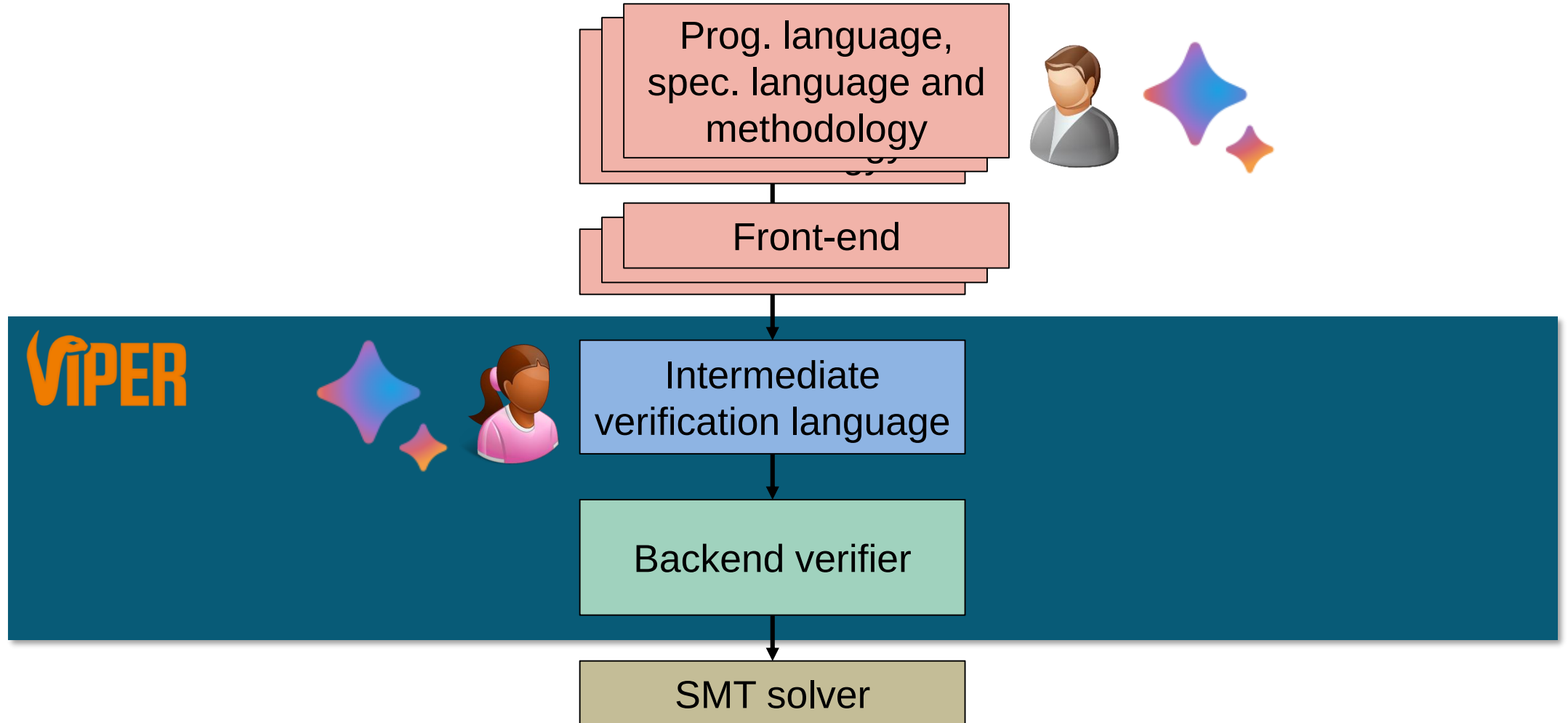
- Goal: verify procedures modularly, that is, independently of their callers
- Equip each procedure with a specification
 - A precondition constrains the parameters
 - A postcondition constrains the results
- Verify that procedure implementation satisfies the specification
- Verify caller against specification
 - Prove that precondition holds
 - Rely on postcondition

```
method indexOf(s: Seq[Int], e: Int)  
returns (res: Int)  
  ensures res < 0 ==> !(e in s)  
  ensures 0 <= res ==> ...  
{  
  ...  
}
```



```
var x: Int  
x := indexOf(Seq(1, 2, 3, 2), 2)  
  
assert x == 1 || x == 3 // succeeds  
assert x == 1           // fails
```







- vipер.ethz.ch
- Try online: <http://vipер.ethz.ch/tutorial>
- Install as VS Code extension

Lecture 1

REASONING ABOUT THE HEAP

Heap model: an object-based language

```
field val: Int

method foo() returns (res: Int)
{
  var cell: Ref
  cell := new(val)
  cell.val := 5
  res := cell.val
}
```

- A heap is a set of objects
- No classes: each object has all fields declared in the entire program
 - Type rules of a source language can be encoded
 - Memory consumption is not a concern since programs are not executed
- Objects are accessed via references
 - Field read and update operations
- No explicit de-allocation (garbage collector)
 - Conceptually, objects could remain allocated

The frame problem

```
field f: Int  
field g: Int
```

```
method set(p: Ref, v: Int)  
  requires p != null  
  ensures p.f == v  
{  
  p.f := v  
}
```

```
x.f := 0  
x.g := 0  
set(x, 5)  
assert x.g == 0
```



- Bad idea: inspect body of callee to determine which field locations are modified
 - Not modular
 - Does not work for abstract methods
- Bad idea: assume conservatively that all field locations may be modified
 - Callee needs a specification for all field locations, even those it does not change
 - Not modular: procedure specifications need to change when a new field is declared

Aliasing

```
field val: Int

method foo(p: Ref, q: Ref)
  requires p != null && q != null
  ensures  p.val == 5
{
  p.val := 5
  q.val := 7
}
```



- Bad idea: require programmer to explicitly specify all non-aliasing properties
 - Does not work for unbounded data structures
 - Violates information hiding

Challenges

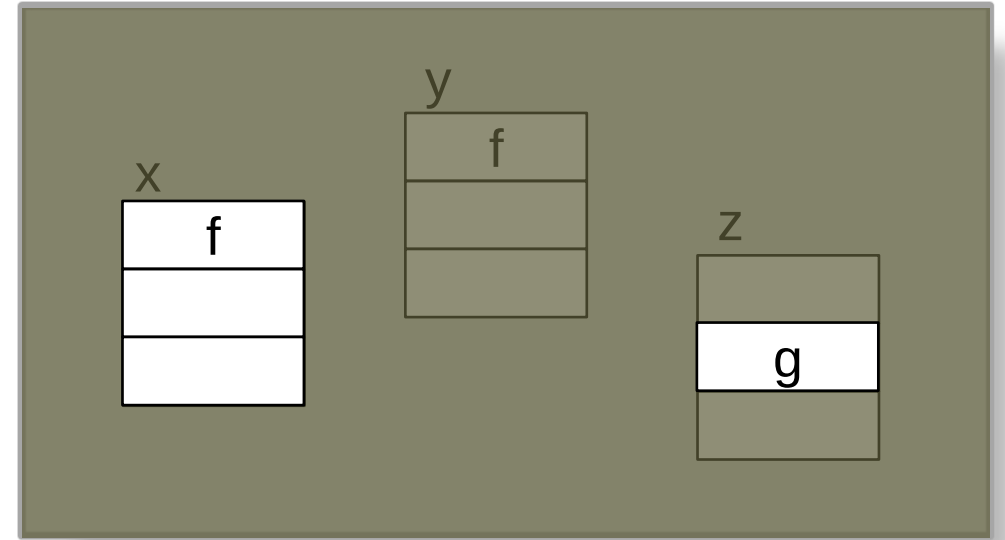
Heap data structures pose three major challenges for sequential verification

- Reasoning about aliasing
- Framing, especially for dynamic data structures
- Writing specifications that preserve information hiding

And additional challenges for concurrent programs, e.g., data races

Access permissions

- Associate each heap location with a permission
- Permissions are created when the heap location is allocated: **new**(f, g)
- Permissions are held by method executions
- Read or write access to a memory location requires permission
- Permissions can be transferred, but not duplicated or forged



x.f := 5



y.f := 5



z.g := x.f



x.f := y.f



Permission assertions

- Permissions are denoted in assertions by **access predicates**
 - Access predicates are not permitted under negations, disjunctions, and on the left of implications
- Assertions may contain both permissions and value constraints
- Many assertions that occur in a program must be **self-framing**, that is, include all permissions to evaluate the heap accesses in the assertion

acc(p.f)

$p.f \mapsto _$

acc(p.f) && p.f > 0

$\exists V \cdot p.f \mapsto V \wedge V > 0$

requires p.f > 0



Separating conjunction

- To handle aliasing, we introduce a new connective: **separating conjunction**
- **$A * B$** holds in a state if:
 - both **A** and **B** hold, and
 - the **sum of the permissions** in **A** and **B** are held in that state
 - **$A * B$** and **$A \wedge B$** are equivalent if **A** and **B** do not contain access predicates
- Holding permission to locations $p.f$ and $q.f$ implies that p and q do not alias
- Viper's **$\&\&$** is separating conjunction
- For the call `swap(x, x)`, the precondition is equivalent to false

$$\text{acc}(p.f) * \text{acc}(q.f) \Rightarrow p \neq q$$

```
method swap(a: Ref, b: Ref)
  requires acc(a.f) && acc(b.f)
```

Framing

Frame rule

$$\frac{\{ \mathbf{A} \} \text{ S } \{ \mathbf{B} \}}{\{ \mathbf{A} * \mathbf{C} \} \text{ S } \{ \mathbf{B} * \mathbf{C} \}}$$

where S does not assign to a local variable that is free in **C**

- The frame **C** must be self-framing
 - If heap locations constrained by **C** are disjoint from those modified by S then **C** is preserved
 - Otherwise, the precondition of the conclusion is equivalent to false (the triple holds trivially)

A typical separation logic proof

```
method set(p: Ref, v: Int)
  requires acc(p.f)
  ensures  acc(p.f) && p.f == v
{
  p.f := v
}
```

```
x := new(f)
y := new(f); y.f := 7

set(x, 5)

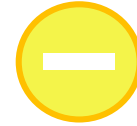
assert x.f == 5 && y.f == 7
```

$$\frac{\frac{\{ \mathbf{acc}(p.f) \} \text{ method set}(p, v) \{ \mathbf{acc}(p.f) * p.f = v \}}{\{ \mathbf{acc}(x.f) \} \text{ set}(x, 5) \{ \mathbf{acc}(x.f) * x.f = 5 \}}}{\{ \mathbf{acc}(x.f) * \mathbf{acc}(y.f) * y.f = 7 \} \text{ set}(x, 5) \{ \mathbf{acc}(x.f) * x.f = 5 * \mathbf{acc}(y.f) * y.f = 7 \}}}$$

Challenges revisited

Heap data structures pose three major challenges for sequential verification

- Reasoning about aliasing
 - Permissions and separating conjunction
- Framing, especially for dynamic data structures
 - Sound frame rule, but no support yet for unbounded data structures
- Writing specifications that preserve information hiding
 - Not solved yet



And additional challenges for concurrent programs, e.g., data races

- Permissions are an excellent basis

Summary Lecture 1

- Verification of heap-manipulating programs poses two main challenges
 - **Aliasing**: Which data structure may overlap in memory?
 - **Framing**: Which heap properties are preserved by a heap modification?
- Separation logic addresses these challenges by two key concepts
 - **Permissions**: All heap accesses are governed by proof obligations that enforce that the current method owns the accessed memory
 - **Separating conjunction**: The conjuncts must hold in disjoint parts of the heap
- These concepts enable separation logic's powerful frame rule

$$\frac{\{ \mathbf{A} \} \text{ S } \{ \mathbf{B} \}}{\{ \mathbf{A} * \mathbf{C} \} \text{ S } \{ \mathbf{B} * \mathbf{C} \}}$$

Lecture 2

ABSTRACTION

Limitations of technique as introduced so far

```
field elem: Int
field next: Ref

method head(this: Ref) returns (res: Int)
  requires acc(this.elem)
  ensures  acc(this.elem)
  ensures  res == this.elem
{
  res := this.elem
}
```

- Specification reveals implementation details

```
method append(this: Ref, e: Int)
  requires // permission to all nodes
  ensures  // list was extended
{
  if(this.next == null) {
    var n: Ref
    n := new(elem, next)
    n.next := null
    this.elem := e
    this.next := n
  } else {
    append(this.next, e)
  }
}
```

- Neither permissions nor behavior can be expressed

Predicates

- User-defined predicates consist of a predicate name, a list of parameters, and a self-framing assertion

```
predicate node(this: Ref) {  
    acc(this.elem) && acc(this.next)  
}
```

- Recursive predicates may denote a statically-unbounded number of permissions

```
predicate list(this: Ref) {  
    acc(this.elem) && acc(this.next) &&  
    (this.next != null ==> list(this.next))  
}
```

Static verification with recursive predicates

- A program verifier in general cannot know statically how far to unfold recursive definitions

```
predicate list(this: Ref) {  
    acc(this.next) &&  
    (this.next != null ==> list(this.next))  
}
```

```
method foo(x: Ref, y: Ref)  
    requires list(x)  
{  
    y.next := null    // do we have permission?  
}
```

Iso-recursive predicates

- An iso-recursive semantics distinguishes between a predicate instance and its body

```
predicate list(this: Ref) {  
  acc(this.elem) && acc(this.next) &&  
  (this.next != null ==> list(this.next))  
}
```

```
method foo(x: Ref, y: Ref)  
  requires list(x)  
{  
  y.next := null // no permission  
}
```



- Intuition: permissions are held by method executions or predicate instances

Folding and unfolding predicates

- Exchanging a predicate instance for its body, and vice versa, is done via **fold and unfold statements** in the program
- An unfold statement exchanges a predicate instance for its body
- A fold statement exchanges a predicate body for a predicate instance

```
method foo(x: Ref)
  requires list(x)
{
  unfold list(x)
  x.next := null
}
```

```
method foo(x: Ref)
  requires list(x)
  ensures list(x)
{
  unfold list(x)
  x.next := null
  fold list(x)
}
```

- Unfolding-expressions allow one to **temporarily unfold** a predicate **during** the evaluation of an expression

Representation invariants

- Data structures typically maintain several consistency conditions
 - Value constraints, e.g., references being non-null or integers being positive
 - Structural constraints, e.g., a tree being balanced
- Such representation invariants are
 - Established by constructors
 - Assumed and preserved by all operations

- Representation invariants can be expressed as part of a predicate

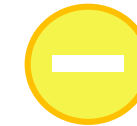
```
predicate list(this: Ref) {  
    acc(this.elem) && acc(this.next) &&  
    (this.next != null ==> list(this.next) &&  
        0 <= this.elem)  
}
```

```
method append(this: Ref, e: Int)  
    requires list(this)  
    ensures list(this)  
{  
    unfold list(this) // assume invariant  
    ...  
    fold list(this) // check invariant  
}
```

Challenges revisited

Heap data structures pose three major challenges for sequential verification

- Reasoning about aliasing
 - Permissions and separating conjunction
- Framing, especially for dynamic data structures
 - Sound frame rule, predicates
- Writing specifications that preserve information hiding
 - Predicates for permissions, not solved for functional behavior



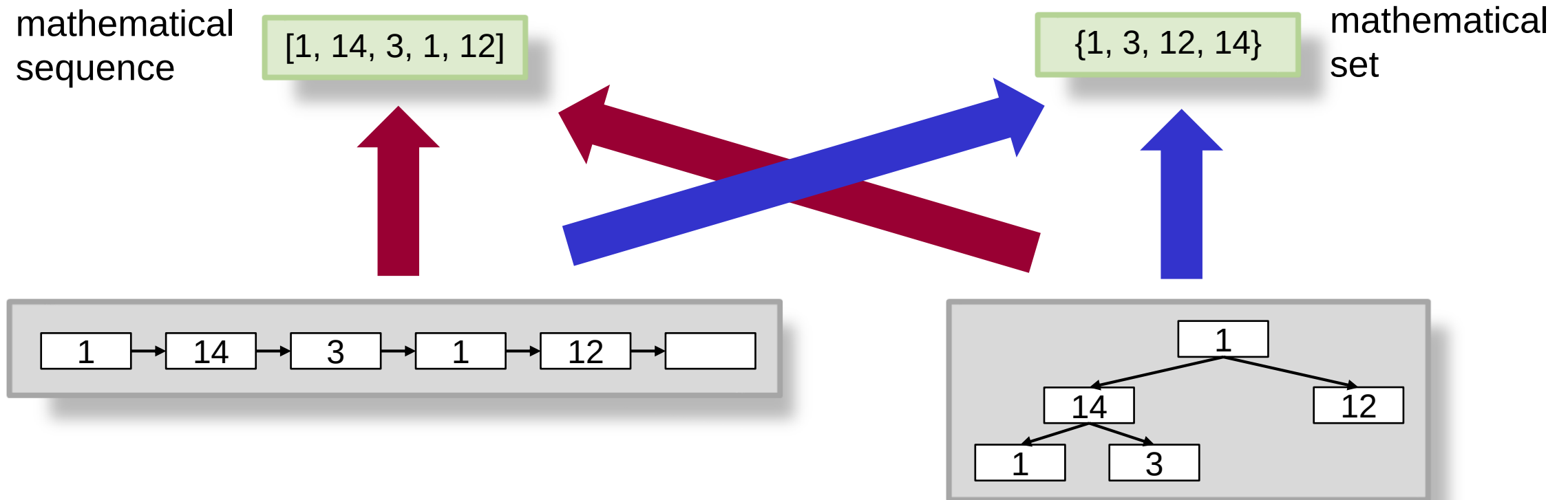
Specifying functional behavior

- Using old-expressions and unfolding-expressions, we can specify some aspects of functional behavior
- But: Approach does not work when behavior depends on an unbounded number of fields (e.g., sorting a list)
- And: specifications reveal implementation details

```
method head(this: Ref) returns (res: Int)
  requires list(this)
  ensures list(this)
  ensures res ==
    old(unfolding list(this) in this.elem)
```

Data abstraction

- To write implementation-independent specifications, we map the concrete data structure to mathematical concepts and specify the behavior in terms of those



Data abstraction via abstraction functions

- Viper provides **heap-dependent functions**

- side-effect free
- terminating
- deterministic

- Function bodies are **expressions**

- Functions may be **recursive**

- Functions must have a **precondition that frames the function body**, that is, provides all permissions to evaluate the body

```
function content(this: Ref): Seq[Int]  
  requires list(this)  
{  
  unfolding list(this) in  
    (this.next == null ?  
      Seq[Int]() :  
      Seq(this.elem) ++ content(this.next)  
    )  
}
```

Challenges revisited

Heap data structures pose three major challenges for sequential verification

- Reasoning about aliasing
 - Permissions and separating conjunction
- Framing, especially for dynamic data structures
 - Sound frame rule, predicates
- Writing specifications that preserve information hiding
 - Predicates, data abstraction, heap-dependent functions



The Viper language

Program code

- Sequential, imperative language
- Standard control structures
- Basic type system
- Built-in heap

Assertion language

- Inductive predicates
- Abstraction functions
- Fractional permissions
- Iterated separating conjunction
- Magic wands

Verification

- Standard contract features
- inhale and exhale
- Permission introspection

Mathematical theories

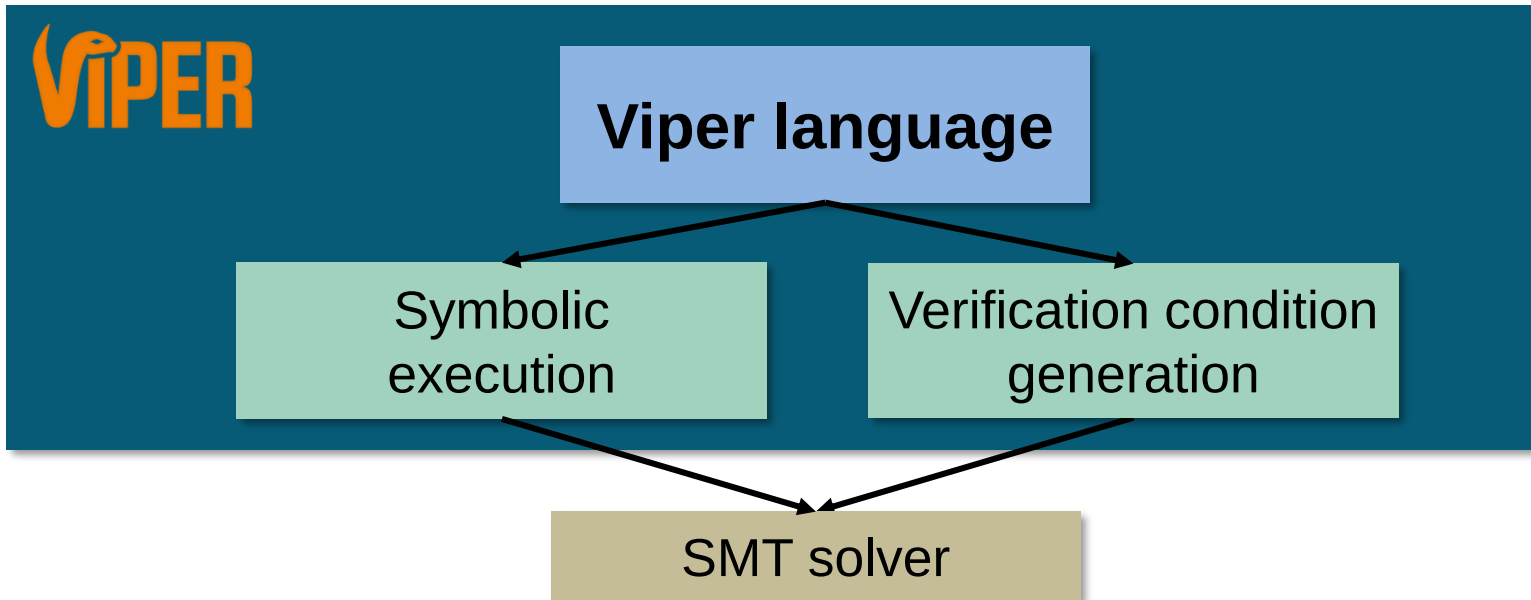
- Predefined datatypes
- User-defined datatypes
- Uninterpreted functions
- Axioms

Summary Lecture 2

- Abstraction mechanisms allow one to write specifications that do not refer to implementation details
 - Specifications of **unbounded data structures**
 - **Information hiding**
 - Specifications of **code that is unavailable** (e.g., abstract methods, foreign functions, binaries)
- Viper offers three main abstraction mechanisms
 - **Methods** abstract over statements
 - **Functions** abstract over expressions
 - **Predicates** abstract over assertions

Lecture 3

AUTOMATING SEPARATION LOGIC



Weakest preconditions

Statement S	$wp \llbracket S \rrbracket (\mathbf{B})$
$x := E$	$\mathbf{B}[E / x]$
$S; S'$	$wp \llbracket S \rrbracket (wp \llbracket S' \rrbracket (\mathbf{B}))$
if $(*) \{ S \}$ else $\{ S' \}$	$wp \llbracket S \rrbracket (\mathbf{B}) \wedge wp \llbracket S' \rrbracket (\mathbf{B})$
assert $\mathbf{A}$	$\mathbf{A} \wedge \mathbf{B}$
assume $\mathbf{A}$	$\mathbf{A} \Rightarrow \mathbf{B}$
havoc x	$\forall x : \mathbf{B}$

To verify a statement S ,
we prove

$$\mathbf{BP} \Rightarrow wp \llbracket S \rrbracket (true)$$

where $\mathbf{BP}$ is a background
predicate encoding properties of
the programming language

Heaps

- Encode references and fields

```
type Ref           // type for references
const null: Ref    // null references

type Field T       // polymorphic type for field names
```

```
field f: Int
field g: Ref
```

```
const f: Field int
const g: Field Ref
```

- Heaps map references and field names to values

```
type HeapType = <T>[Ref, Field T]T    // polymorphic map
```

- Represent the program heap as global variable

```
var Heap: HeapType
```

Permissions and field access

- Permissions are tracked in a global permission mask

```
type MaskType = <T>[Ref, Field T]bool  
var Mask: MaskType
```

- Convention: $\neg \text{Mask}[\text{null}, f]$ for all fields f
- Field access

```
v := x.f
```

```
assert Mask[x,f]  
v := Heap[x,f]
```

```
x.f := E
```

```
assert Mask[x,f]  
Heap[x,f] := E
```

- Field access requires permission!
- Permission check guarantees absence of null-dereference

Encoding into guarded commands: conditionals

- Other statements can be encoded into guarded commands
- For example, conditional statements

```
if ( E ) { S } else { S' }
```

can be encoded using nondeterministic choice and assume

```
[[ if ( E ) { S } else { S' } ]] = if ( * ) { assume E; S } else { assume ¬E; S' }
```

A typical separation logic proof

```
method set(p: Ref, v: Int)
  requires acc(p.f)
  ensures  acc(p.f) && p.f == v
{
  p.f := v
}
```

```
x := new(f)
y := new(f); y.f := 7

set(x, 5)

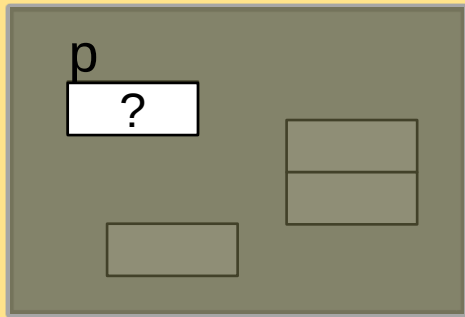
assert x.f == 5 && y.f == 7
```

$$\frac{\frac{\{ \mathbf{acc}(p.f) \} \text{ method set}(p, v) \{ \mathbf{acc}(p.f) * p.f = v \}}{\{ \mathbf{acc}(x.f) \} \text{ set}(x, 5) \{ \mathbf{acc}(x.f) * x.f = 5 \}}}{\{ \mathbf{acc}(x.f) * \mathbf{acc}(y.f) * y.f = 7 \} \text{ set}(x, 5) \{ \mathbf{acc}(x.f) * x.f = 5 * \mathbf{acc}(y.f) * y.f = 7 \}}}$$

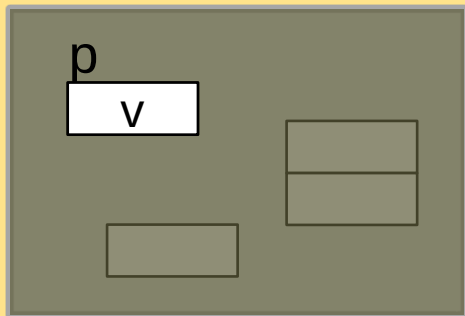
Automating a separation logic proof

```
method set(p: Cell, v: Int)
  requires acc(p.f)
  ensures  acc(p.f) && p.f == v
```

```
{
```

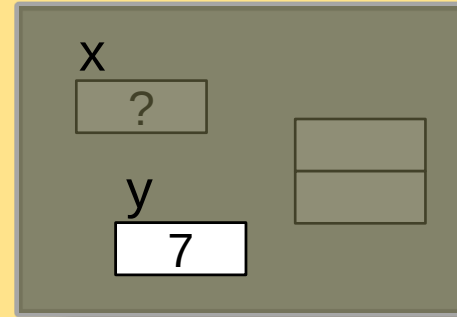


```
  p.f := v
```



```
}
```

```
x := new(f); x.f := 2
y := new(f); y.f := 7
```



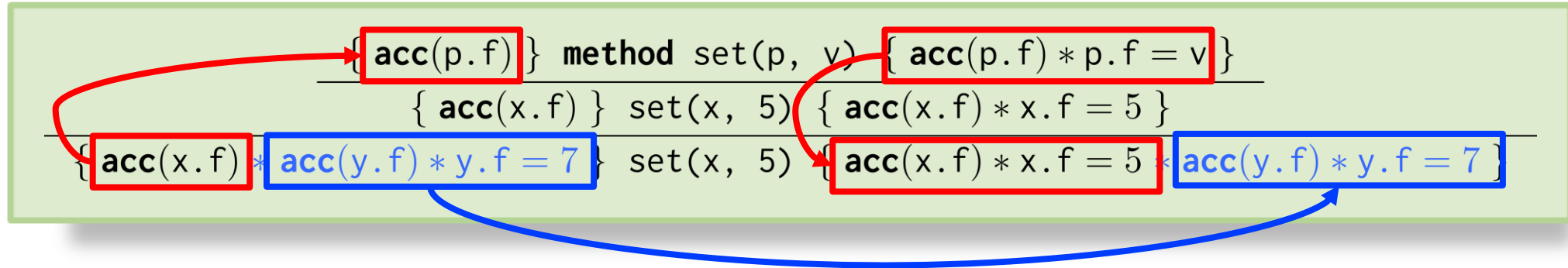
```
set(x, 5)
```

5

```
assert x.f == 5 && y.f == 7
```

Framing!

Permission transfer for method calls



- Calling a method **transfers permissions from the caller to the callee** (according to the method precondition)
- Returning from a method **transfers permissions from the callee to the caller** (according to the method postcondition)
- **Residual permissions are framed around the call**

Reasoning about loops

```
x := new(f); x.f := 0
y := new(f); y.f := 7

while (x.f < 10)
  invariant acc(x.f)
{
  x.f := x.f + 1
}
assert y.f == 7
```

$$\frac{\frac{\{ \text{acc}(x.f) \} \text{ while}(x.f < 10) \{ \dots \} \{ \text{acc}(x.f) \} \quad \{ \text{acc}(x.f) \} \text{ while}(x.f < 10) \{ \dots \} \{ \text{acc}(x.f) * \neg x.f < 10 \} }{\{ \text{acc}(x.f) * \text{acc}(y.f) * y.f = 7 \} \text{ while}(x.f < 10) \{ \dots \} \{ \text{acc}(x.f) * \neg x.f < 10 * \text{acc}(y.f) * y.f = 7 \} }$$

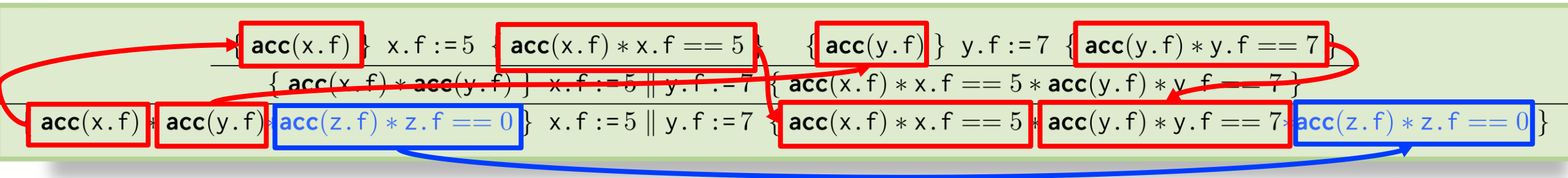
Reasoning about parallel branches

```
x := new(f); x.f := 0
y := new(f); y.f := 0
z := new(f); z.f := 0
```

```
requires acc(x.f)
ensures  acc(x.f) && x.f == 5
x.f := 5
```

```
requires acc(y.f)
ensures  acc(y.f) && y.f == 7
y.f := 7
```

```
assert x.f + y.f + z.f == 12
```



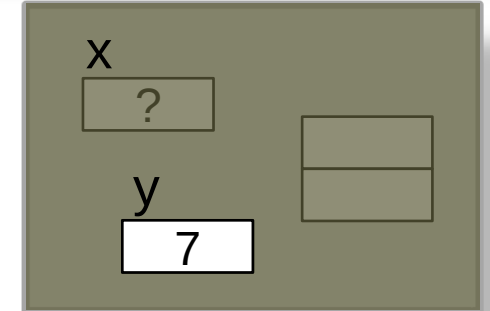
Permission transfer: inhale and exhale operations

- **inhale** **A** means:

- obtain all permissions required by assertion **A**
- assume all logical constraints

```
inhale acc(x.f) && x.f == 2
```

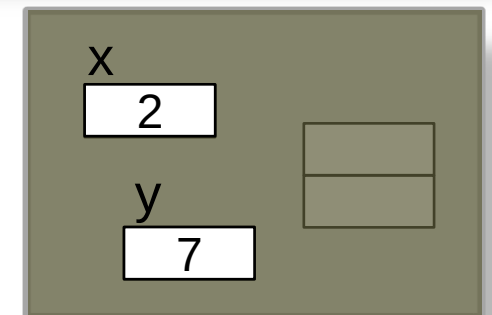
2



- **exhale** **A** means:

- assert all logical constraints
- check and remove all permissions required by assertion **A**
- havoc any locations to which all permission is lost

```
exhale acc(x.f) && x.f == 2
```



Inhale

- **inhale A** means:
 - obtain all permissions required by assertion **A**
 - assume all logical constraints
- Encoding is defined recursively over the structure of **A**

inhale E

assume $[[E]]$

inhale acc(E.f)

assume $\neg \text{Mask}[[[E]], f]$
 $\text{Mask}[[[E]], f] := \text{true}$

Reaching more than full permission goes to magic

inhale E => A

if($[[E]]$) { **[[inhale A]]** }

inhale A && B

[[inhale A]]; **[[inhale B]]**

Separating conjunction:
add sum of permissions

- The encoding also asserts that E is well-defined (omitted here)

Exhale (simplified)

- **exhale A** means:
 - assert all logical constraints
 - check and remove all permissions required by assertion **A**
 - havoc any locations to which all permission is lost
- Encoding is defined recursively over the structure of **A**

exhale E

assert $[[E]]$

exhale acc(E.f)

assert $\text{Mask}[[[E]], f]$
 $\text{Mask}[[[E]], f] := \text{false}$
havoc $\text{Heap}[[[E]], f]$

exhale E => A

if($[[E]]$) { **[[exhale A]]** }

exhale A && B

[[exhale A]]; **[[exhale B]]**

Separating conjunction:
remove sum of permissions

- The encoding also asserts that E is well-defined (omitted here)

Simplified proof obligations for method bodies and calls

```
method foo() returns (...)  
  requires A  
  ensures B  
  { S }
```

```
x := foo()
```

■ Encoding *without heap*

- Body

```
assume A  
// encoding of S  
assert B
```

- Call

```
assert A[...]  
havoc x  
assume B[...]
```

■ Encoding *with heap*

- Body

```
inhale A  
// encoding of S  
exhale B
```

- Call

```
exhale A[...]  
havoc x  
inhale B[...]
```

■ **inhale** and **exhale** are permission-aware analogs of **assume** and **assert**

Encoding of predicates

- Recall that permissions are tracked in a global permission mask

```
type MaskType = <T>[Ref, Field T]bool  
var Mask: MaskType
```

- We use the same mask to track predicate instances
- An unfold statement exchanges a predicate instance for its body
- A fold statement exchanges a predicate body for a predicate instance

unfold $P(\bar{E})$

exhale $P(\bar{E})$

inhale $body(P(\bar{E}))$

fold $P(\bar{E})$

exhale $body(P(\bar{E}))$

inhale $P(\bar{E})$

Encoding of heap-dependent functions

- Heap-dependent functions are encoded as uninterpreted functions
- Function body is encoded as a **definitional axiom**

```
function f(x: T): T' {  
  E  
}
```

```
function f(x: T, h: HeapType): T'  
axiom forall x: T, h: HeapType :: f(x, h) == [[E]]
```

(will be revised later)

- `[[_]]` is the encoding function (omitted for types), parametric in the heap
 - A proof obligation checks that the function body is well-defined (omitted here)
- Function calls are encoded as applications of these functions

```
f(E)
```

```
f([[E]], Heap)
```

Another frame problem

```
function content(this: Ref): Seq[Int]
  requires list(this)
{
  this.next == null ?
    Seq[Int]() :
    Seq(this.elem) ++ content(this.next)
}
```

```
// assume we have list(x) && acc(y.f)
tmp := content(x)
y.f := 5
assert tmp == content(x)
```



```
tmp := content(x, Heap)
assert Mask[y, f]
Heap[y, f] := 5
assert tmp == content(x, Heap)
```

- Each heap update modifies the (global) heap
- Any information about heap-dependent functions is lost
- Recovering the information by inspecting the function body would violate information hiding and would not work for abstract functions

Framing axioms

- The function precondition specifies a read effect, which is used to generate a framing axiom for the function
- If two heaps agree on a function's read effect then the function yields the same result in both heaps

```
function get(x: Ref): Int
  requires acc(x.elem)
{ ... }
```

```
function get(x: Ref, h: HeapType): int

axiom forall x: Ref, h1: HeapType, h2: HeapType ::
  h1[x,elem] == h2[x,elem] ==> get(x, h1) == get(x, h2)
```

Actual axiom is more complex to break symmetry,
which causes unnecessary quantifier instantiations

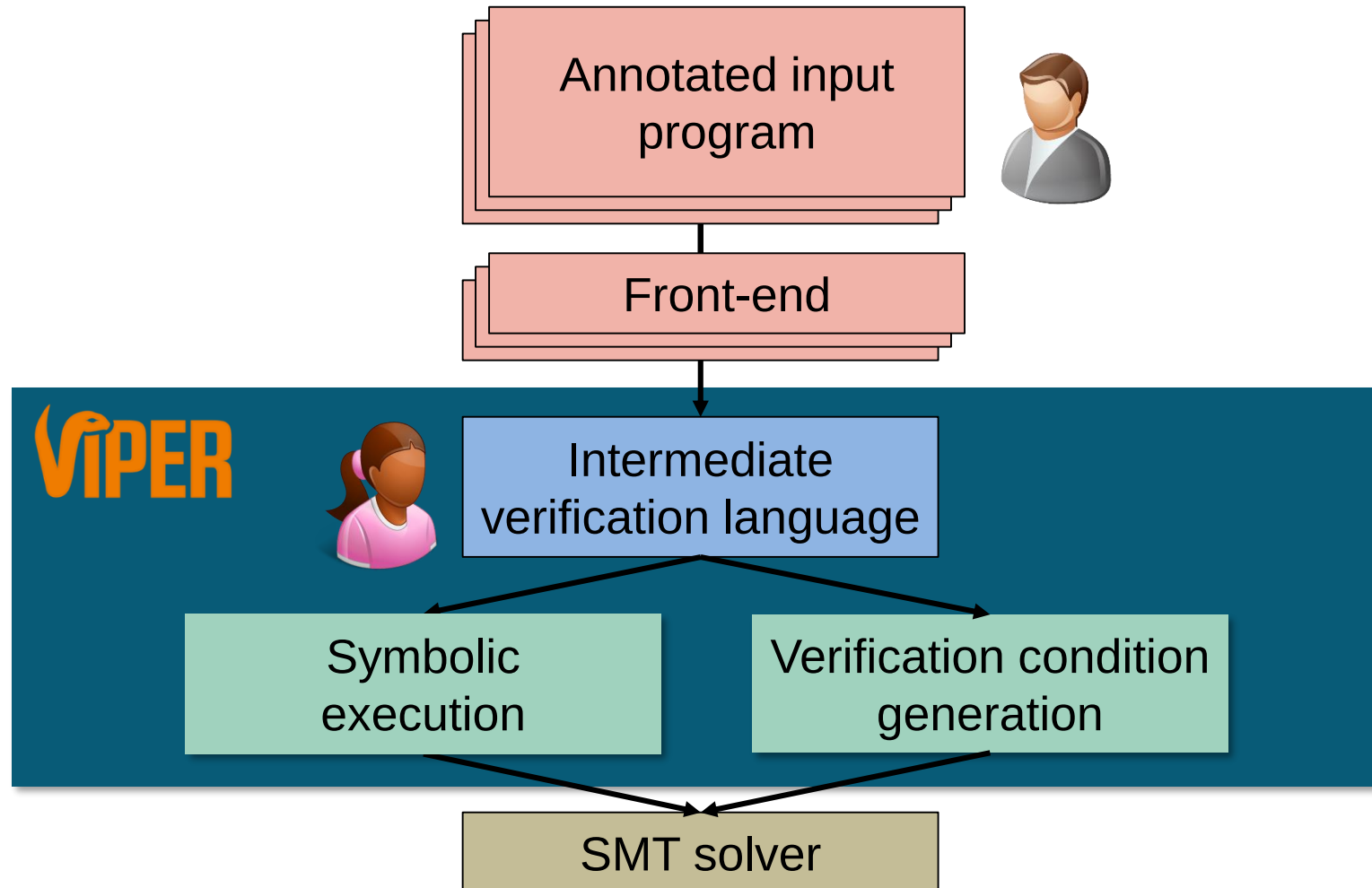
- The encoding for predicates in function preconditions is analogous, but needs to consider all heap locations included in a predicate

Summary Lecture 3

- One way to compute proof obligations for imperative programs is encoding them into guarded commands and applying a weakest-precondition transformer
 - **assert statements**: What properties must be proved?
 - **assume statements**: What properties may be used in these proofs?
- Proof rules in separation logic are combinations of **permission transfers**
 - Any properties from the pre-state that are not transferred away are **framed**
- Viper offers primitive statements for permission transfers, which allow one to encode a wide range of separation logic rules
 - **exhale statements** assert logical conditions and remove permissions
 - **inhale statements** assume logic conditions and add permissions
- **inhale** and **exhale** are permission-aware analogs of **assume** and **assert**

Lecture 4

FRONTENDS



Frontend encodings

	Input language	Viper language
State model	▪ Pointers ▪ Arrays, slices ▪ Type system	▪ Object-based ▪ Semantic model ▪ Logical encoding
Execution model	▪ Complex statements ▪ Higher-order computations ▪ Effects (exceptions, I/O)	▪ Simple statements ▪ Defunctionalization ▪ Encoding
Program logic	▪ Concurrency ▪ Higher-order	▪ Universal primitives ▪ Defunctionalization

Go verification in Gobra

```
requires isList(a) && isList(b)
func client(a *list, b *list) {
    tmp := b.length()
    a.appendBack(100)
    assert b.length() == tmp
}
```

The Gobra logo, featuring the word "gobra" in a stylized orange font with a small orange bird-like icon above the 'g'.

Functional properties

Memory errors

Aliasing

Data races

- To handle these challenges, Gobra uses a separation logic similar to Viper's
- Many existing verifiers take this approach because it enables modular verification of programs in mainstream languages, including heap-manipulating and concurrent programs

Example: Go verification in Gobra

```
requires acc(x) && acc(y)
ensures  acc(x) && acc(y)
ensures  *x == old(*y)
ensures  *y == old(*x)
func swap(x *int, y *int) {
    tmp := *x
    *x = *y
    *y = tmp
}
```



- Go supports pointers to integers
- Parameters can be assigned to
- Locals get initialized by default

```
field val: Int
```

```
method swap(x: Ref, y: Ref)
    requires acc(x.val) && acc(y.val)
    ensures  acc(x.val) && acc(y.val)
    ensures  x.val == old(y.val)
    ensures  y.val == old(x.val)
{
    var yLocal: Ref // declare locals
    var xLocal: Ref

    xLocal := x      // copy parameters
    yLocal := y

    var tmp: Int     // declare tmp
    inhale tmp == 0

    tmp := xLocal.val // tmp = *x
    xLocal.val := yLocal.val // *x = *y
    yLocal.val := tmp      // *y = tmp
}
```

Exposing the verification logic

```
requires acc(x) && acc(y)
ensures  acc(x) && acc(y)
ensures  *x == old(*y)
ensures  *y == old(*x)
func swap(x *int, y *int) {
    tmp := *x
    *x = *y
    *y = tmp
}
```

The logo for Gobra, featuring the word "gobra" in a stylized orange font with a small flame-like shape above the 'g'.

- Developers need to use separation logic
- The overhead for programmers is substantial (both amount and complexity of annotations)

Rust and its type system

```
fn client(a: &mut List, b: &mut List)
{
    let tmp = b.length();
    a.appendBack(100);
    assert!(b.length() == tmp);
}
```



Functional properties

No memory errors

Controlled aliasing

No data races

Can we leverage this type system to simplify verification?

Example: Rust verification in Prusti

```
#[ensures(*x == old(*y) )]  
#[ensures(*y == old(*x) )]  
fn swap(x: &mut i32, y: &mut i32) {  
    let tmp = *x;  
    *x = *y;  
    *y = tmp;  
}
```

- Prusti extracts permissions automatically from type information
- A Viper “core proof” of memory safety is generated completely automatically
- Users can add **functional correctness** specifications, by using a slight extension of Rust expressions

The overhead for programmers is substantially reduced
(both amount and complexity of annotations)

Comparison of annotation overhead

```
#[feature(box_patterns)]

use prusti_contracts::*;

struct Node {
    elem: i32,
    next: List,
}

enum List {
    Empty,
    More(Box<Node>),
}

impl List {
    #[pure]
    #[ensures(result >= 0)]
    fn len(&self) -> usize {
        match self {
            List::Empty => 0,
            List::More(box node) =>
                1 + node.next.len(),
        }
    }

    #[ensures(result.len() ==
        self.len() + that.len())]
    pub fn zip(&self, that: &List) -> List {
        match self {
            List::Empty => that.cloneList(),
            List::More(box node) => {
                let new_node = Box::new(Node {
                    elem: node.elem,
                    next: that.zip(&node.next),
                });
                List::More(new_node)
            }
        }
    }
}
```

```
#[ensures(result.len() == self.len())]
pub fn cloneList(& self) -> List {
    match self {
        List::Empty => List::Empty,
        List::More(box node) => {
            let new_node = Box::new(Node {
                elem: node.elem,
                next: node.next.cloneList(),
            });
            List::More(new_node)
        }
    }
}
```

P*rust → i

```
field next: Ref
field elem: Int

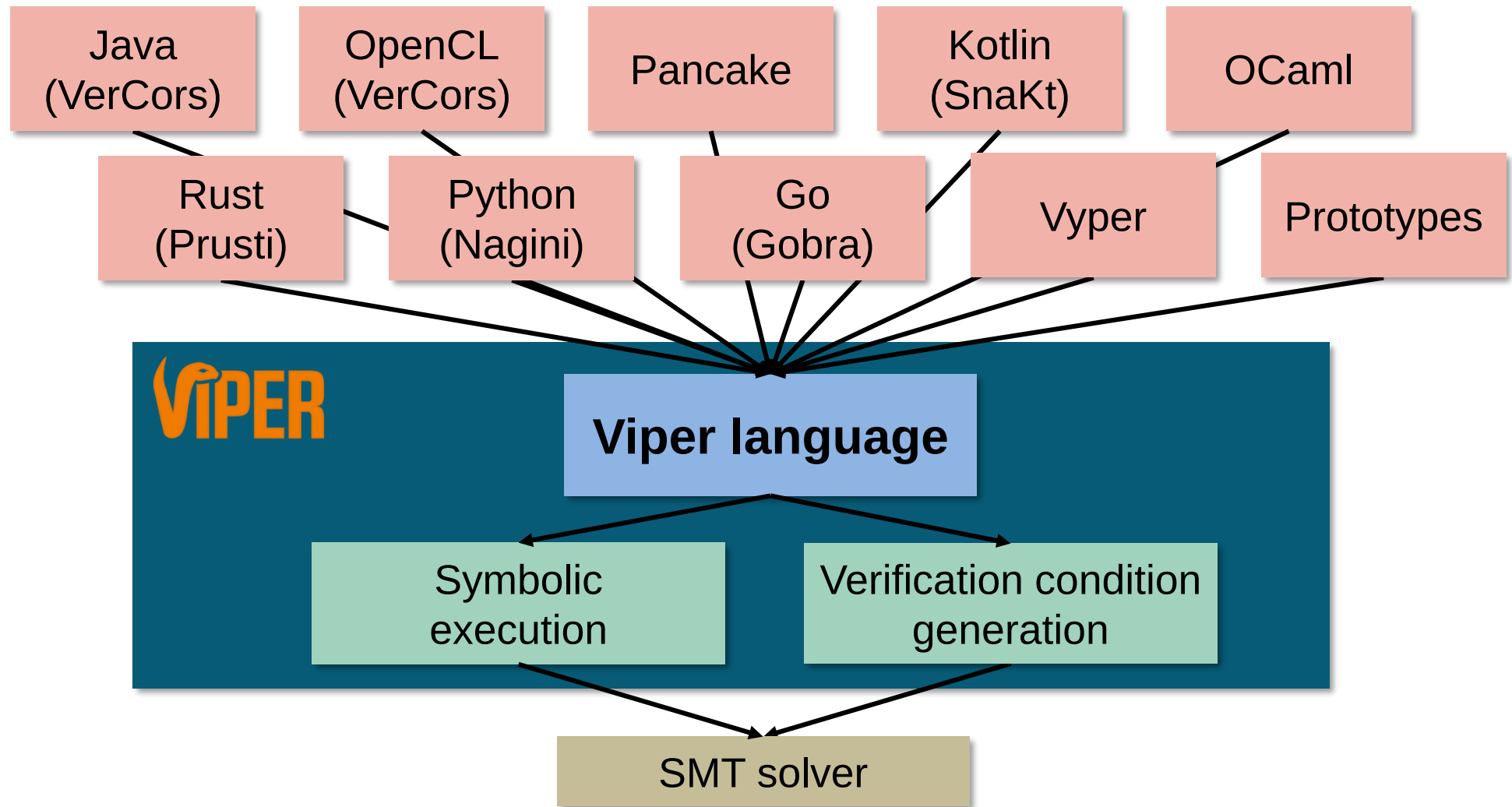
predicate list(this: Ref) {
    acc(this.elem) && acc(this.next) &&
    (this.next != null => list(this.next))
}

function len(this: Ref): Int
    requires acc(list(this), wildcard)
{
    unfolding acc(list(this), wildcard) in
    (this.next == null ? 0 : len(this.next) + 1)
}

method zip(this: Ref, that: Ref)
    returns (res: Ref)
    requires acc(list(this), 1/2) &&
        acc(list(that), 1/2)
    ensures acc(list(this), 1/2) &&
        acc(list(that), 1/2)
    ensures list(res)
    ensures res != null
    ensures len(res) == len(this) + len(that)
{
    unfold acc(list(this), 1/2)
    if(this.next == null) {
        res := cloneList(that)
    } else {
        res := new(*)
        res.elem := this.elem
        var rest: Ref
        rest := zip(that, this.next)
        res.next := rest
        fold list(res)
    }
    fold acc(list(this), 1/2)
}
```

```
method cloneList(this: Ref) returns (res: Ref)
    requires acc(list(this), 1/2)
    ensures acc(list(this), 1/2) && list(res)
    ensures res != null
    ensures len(res) == len(this)
{
    res := new(*)
    unfold acc(list(this), 1/2)
    if(this.next == null) {
        res.next := null
    } else {
        var tmp: Ref
        tmp := cloneList(this.next)
        res.elem := this.elem
        res.next := tmp
    }
    fold acc(list(this), 1/2)
    fold list(res)
}
```

VIPER



Diversity of existing Viper frontends

Language features

- Imperative and object-oriented code
- Nominal and structural typing
- Ownership types
- Diverse state models
- Multithreading with shared state and message passing
- Weak-memory concurrency
- I/O operations
- Closures and iterators
- Digital assets

Properties

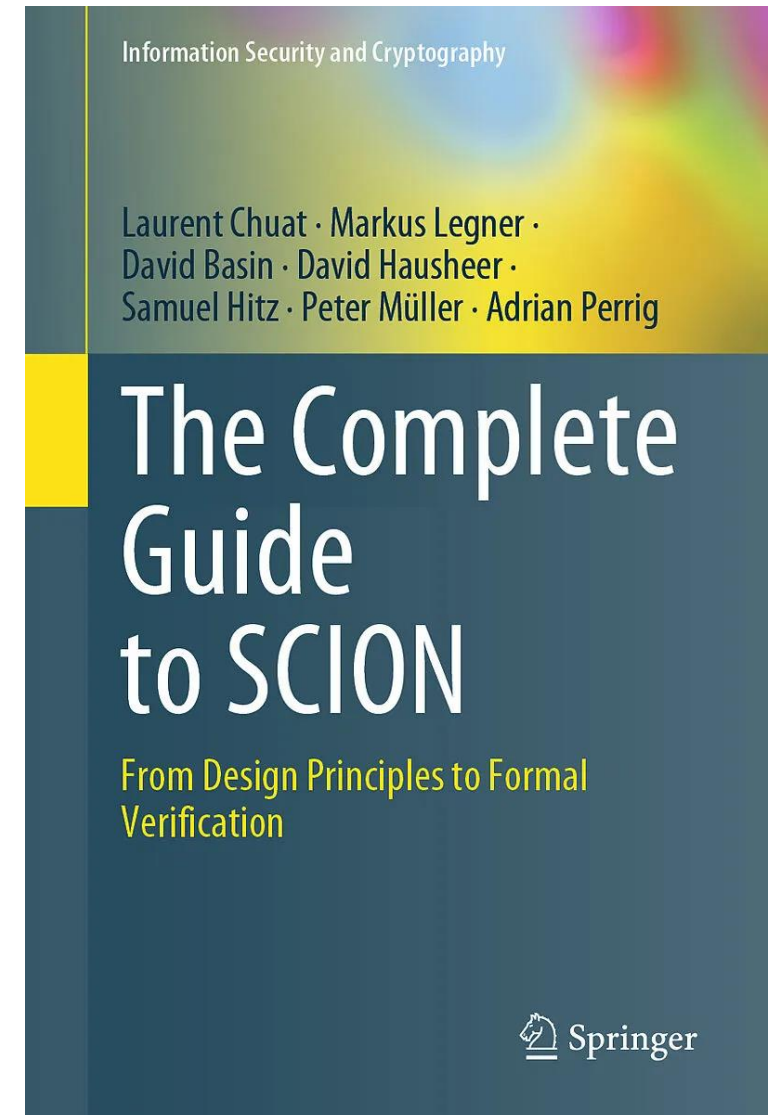
- Memory safety
- Crash freedom
- Absence of overflows
- Termination
- Functional correctness
- Race freedom
- Deadlock freedom
- Secure information flow
- Asset manipulation
- Worst-case execution time

Limitations

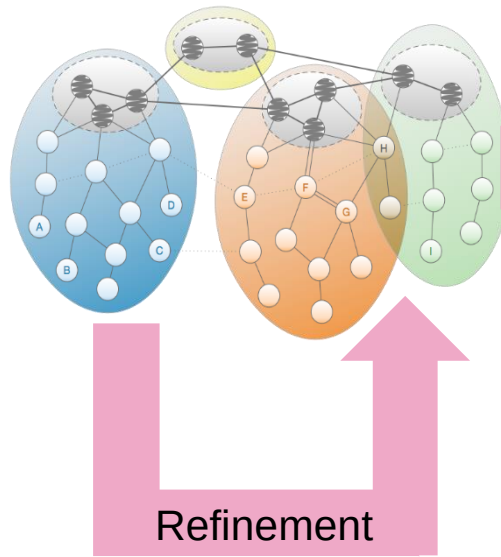
- Limitations inherited from the SMT solver
 - Undecidable theories may lead to spurious errors
 - Verification time for large methods
- Annotation overhead
 - Typically 2-5 lines of annotations per line of code
- Trust assumptions
 - Correctness of SMT solver
 - Correctness of Viper
 - Correctness of front-end encoding

Scion Internet Architecture

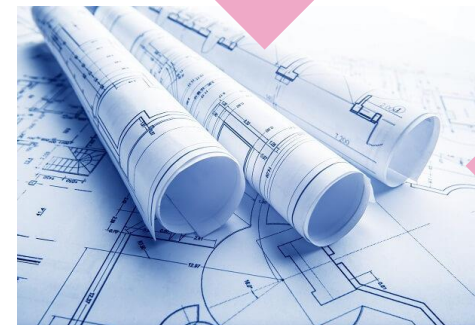
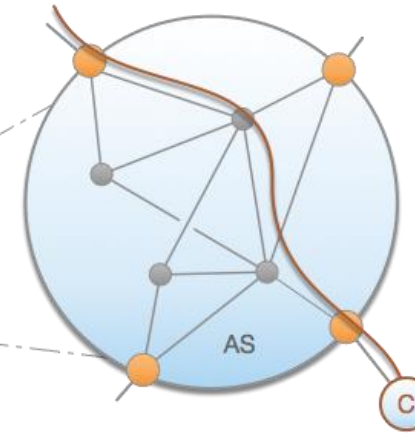
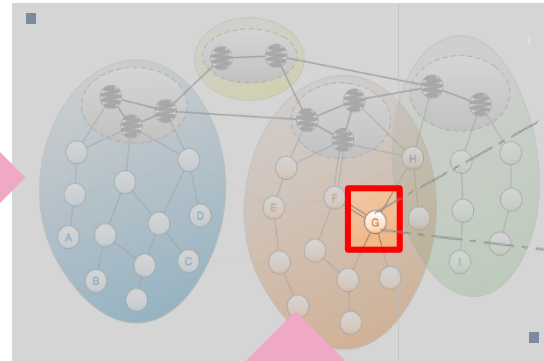
- Scion is a new architecture for inter-domain routing
- Research and commercial deployments
- Our goal: formal end-to-end verification of security and correctness
- Protocol
 - Secure packet forwarding
- Code
 - Crash freedom
 - Functional correctness
 - Termination
 - Protocol compliance



Mathematical model
of entire network



Mathematical model
of border router



Router specification

Verification



Router implementation

Results of SCION router verification

- Completed verification of SCION router (4,700 lines of optimized Go)
 - Crash freedom
 - Functional correctness
 - Termination
 - Protocol compliance
- 16,700 lines of annotations (3.6 LoS per LoC)
- Required only three code changes
- Identified 14 confirmed issues related to memory safety, functional correctness, and protocol compliance (plus 2 performance issues)
- Despite extensive code reviews, testing, and fuzzing
- Triggered various performance optimizations

Summary Lecture 4

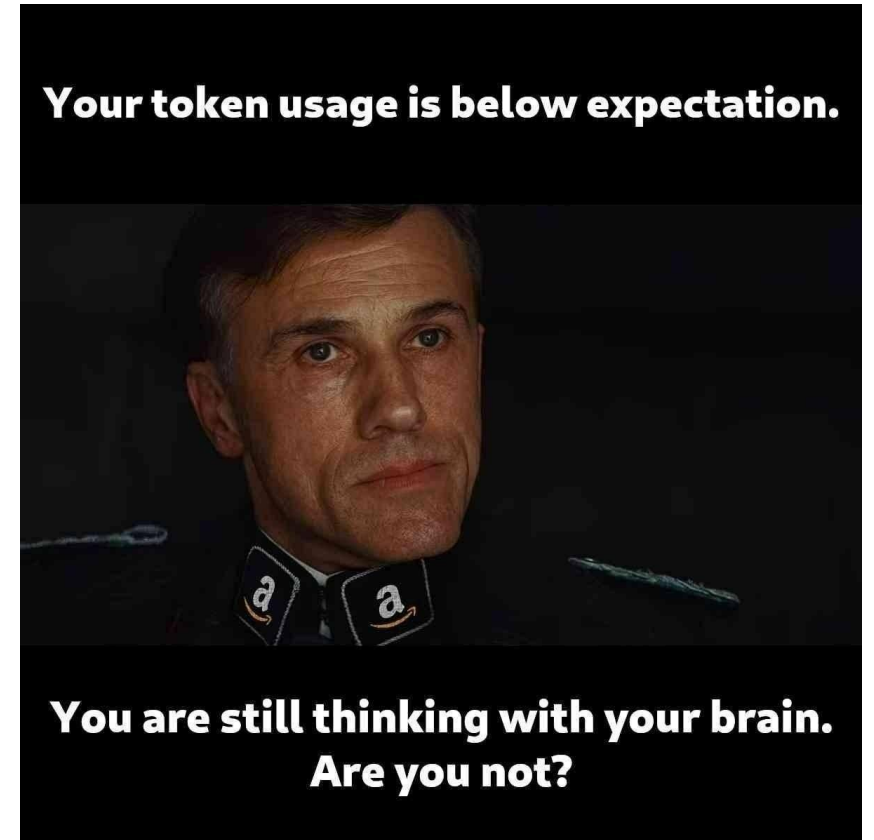
- The concepts supported by Viper are sufficiently expressive to encode a wide range of verification problems for **different languages, programs, and properties**
- Viper powers many powerful program verifiers
 - Most of them reflect Viper's verification logic directly
 - Some aim to reduce the annotation overhead by leveraging ownership type systems
- Some of these verifiers have been used for **substantial verification efforts** in academia and industry
 - Annotation overhead and performance on large codebases remain challenges

Conclusions

RESEARCH DIRECTIONS

AI amplifies the demand for verification

- Large amounts of code get generated by agents
- This code contains a significant number of bugs and security vulnerabilities
- Code reviews have become the main bottleneck of software development
- At the same time, AI has become extremely effective in detecting vulnerabilities

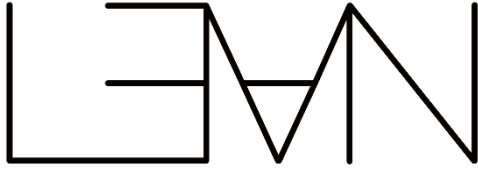


Both trends will lead to an increased demand for program verification

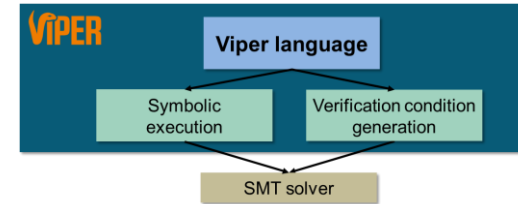
Research direction 1: program logics

- Program logics capture **essential correctness arguments** for programs
 - Invariants to reason about loops
 - Ranking functions to prove termination
 - Lock orders to prove deadlock freedom
- Just like human developers, AI agents **need logics to phrase proofs**
- Developing logics for advanced **programs and properties** poses many open research problems:
 - Probabilistic programs and properties
 - Hyperproperties
 - Distributed systems
 - Quantum computing

Research direction 2: agentic verification

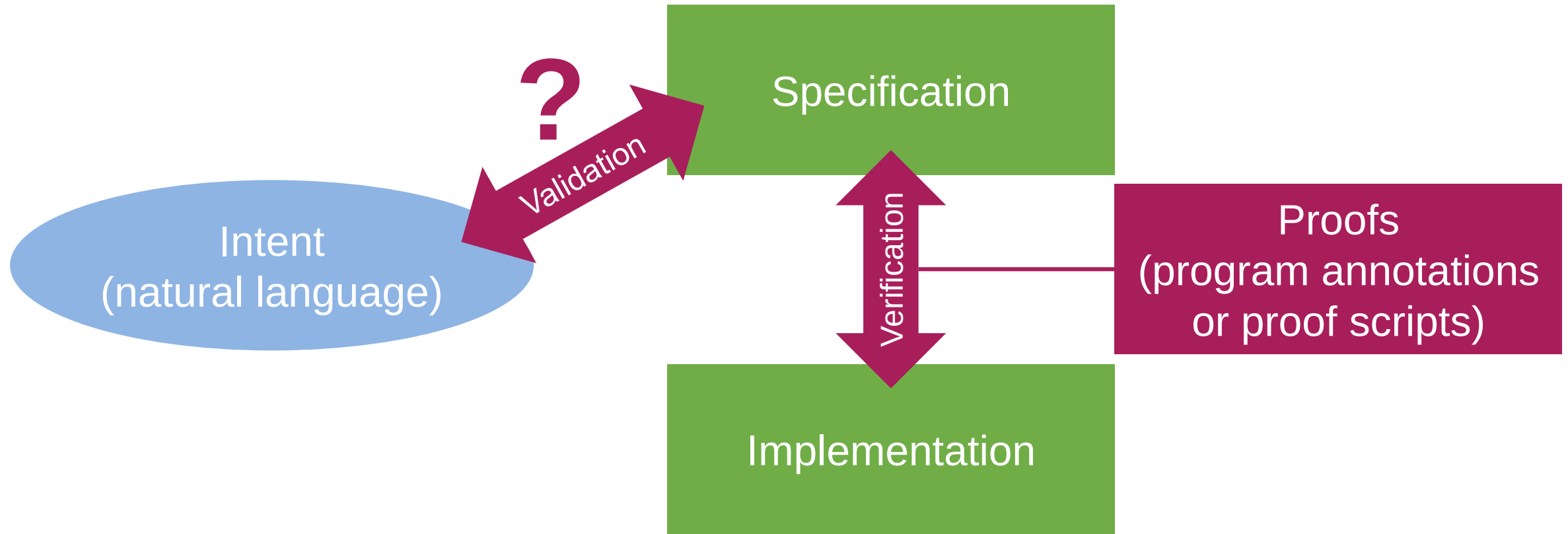


- Explicit proof objects might be better training targets
- Expressiveness is not limited by an SMT solver
- User interaction typically exposes complexity of underlying logic



- Annotated programs cater to the strengths of LLMs, which are good at coding
- Relying primarily on an SMT solver is likely to be faster and cheaper
- Code-centric approach might facilitate interactions between developers and AI (vericoding)

Research direction 3: validation



How to validate AI-generated specifications and code?